COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/15/26	**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L. Santos, Adrian Walter D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **[Hands-on Activity 13.1 Parallel Algorithms and Multithreading]**

1. PROCEDURE OUTPUT

ILO A & B

PART 1 OUTPUT

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis (Reyes)	The program starts by getting a thread to execute print 10 and T.I.P, waits for the threat to complete, then printing both values at the same time (the output says otherwise as each thread output has different execution times)
Analysis (Santos)	The driver function uses thread from the library <thread> in order to print out the contents of the callable print. The program then used t1.join in order to join the lines inside the callable which then executes the printing process which resulted in both lines being printed

PART 2 OUTPUT

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {    'vector' is not a member of 'std'
    std::vector<std::string> s = {    no instance of constructor "std::vector<_Ty, _Alloc>::vector [with _Ty
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };    'vector' is not a member of 'std'
    std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++) {    class "std::vector<std::string, std::allocator<std::string>>" ha
        threads.push_back(std::thread(print, i, s[i]));    class "std::vector<std::thread, std::allocator<st
    }
    for (auto &th : threads) {    this range-based 'for' statement requires a suitable "begin" function and
        th.join();
    }
    return 0;
}
```

```
1 : Competent0 : T.I.P.2 : Computer

3 : Engineers

PS C:\Users\TIPQC\Downloads\Thread>
```

Analysis (Reyes)	What I'm seeing is due to the unsynchronized threads trying to execute print,
Analysis (Santos)	This code tries to run four separate tasks at the same time to print out a list of words with their numbers. Because all four tasks print to the screen at the exact same time without waiting their turn, the printed words overlap and mix together.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference

Questions:
1. Write a definition of multithreading and its advantages/disadvantages.

Answer (Reyes): Multithreading is allowing a program to use multiple threads to enhance its performance. The advantages of using multiple threads is improved performance due to the use of multiple cores and better responsiveness. The cost of using multithreading is that they're unsynchronized, meaning you have to manage their outputs, it also consumes more memory because each thread that you use needs its own stack.	Answer (Santos): Multithreading is executing multiple threads simultaneously within a single program too. The advantages of using multiple threads are boosted execution speed. The cost of using multithreading is figuring out the complexity of managing multiple threads and the space it takes up.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Answer (Reyes): Most web servers use multithreading because they're just too many concurrent (overlapping) request happening simultaneously, therefore they would prefer the use of multithreading.	Answer (Santos): Multithreading lets apps run heavy tasks in the background so the main screen never freezes. It also helps speed up work like loading images or downloading files by doing them all at once.

3. Differentiate between parallelism and concurrency.

Answer (Reyes):	Answer (Santos): Concurrency is when a computer quickly switches back and forth between different tasks so they all make progress together on one core. Parallelism is when a multi-core computer actually runs those different tasks at the exact same time side by side.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
Display the value of global variable at different combinations of using the join() per thread. Such that:
o Display

o T1.join()
o Display
o T2.join()
o Display
o T3.join()
Provide an analysis based on the outputs of the multi-threading exercise.

INPUT

```cpp
#include <iostream>
#include <thread>

int total = 0;
void add(int value) {
    total += value;
}

int main() {

    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

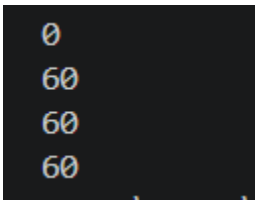
    std::cout << total << std::endl;
    T1.join();

    std::cout << total << std::endl;
    T2.join();

    std::cout << total << std::endl;
    T3.join();

    std::cout << total << std::endl;

    return 0;
}
```

OUTPUT


Analysis	
Reyes	When it shows the number count before waiting for a thread, there will be inaccurate results due to concurrent background tasks. Using join() ensures all tasks complete first, guaranteeing an accurate final count.
Santos	Showing the number before waiting for a thread can give wrong results because tasks run at the same time in the background. Using join() forces the program to wait until each task is done, making sure the final number is correct.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

INPUT

```cpp
#include <iostream>
#include <vector>
#include <thread>


int foundIndex = -1;

void linearSearch(const std::vector<int>& arr, int target, int start, int end) {
    for (int i = start; i < end; ++i) {
        if (arr[i] == target) {
            foundIndex = i;
            return;
        }
    }
}

int main() {
    std::vector<int> numbers = {5, 12, 8, 25, 3, 19, 42, 7, 31, 14};
    int target = 42;
    int mid = numbers.size() / 2;


    std::thread T1(linearSearch, std::cref(numbers), target, 0, mid);
    std::thread T2(linearSearch, std::cref(numbers), target, mid, numbers.size());

    T1.join();
    T2.join();

    if (foundIndex != -1) {
        std::cout << "Target " << target << " found at index " << foundIndex << std::endl;
    } else {
        std::cout << "Target " << target << " not found." << std::endl;
    }

    return 0;
}
```

OUTPUT	
Target 42 found at index 6	
Analysis (Reyes)	Splitting a list across multiple threads lets linear search process both halves at once, reducing search time. However, running these parallel threads uses extra system memory.
Analysis (Santos)	By using multithreading on linear search the list gets split in half which makes the time complexity better as each half can simultaneously be searched at the same time. This makes linear search more efficient but with the disadvantage of taking more space for the device for it to run.

Conclusion (Reyes)	Multithreading helps by dividing large tasks across threads to allow simultaneous data processing, speeding up execution through a divide-and-conquer approach. While this boosts algorithm performance, it trades off higher memory consumption and requires capable hardware.
Conclusion (Santos)	Multithreading improves algorithm performance by splitting big tasks across multiple threads to process data at the same time. Overall it improves the speed and efficiency of algorithms as it follows a divide and conquer principle as the tasks are split with the cost of consuming more space. Its applications can span a wide variety of uses as long as the device has the capability to compensate for the increase in space.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.